



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN GENERAL DE PREGRADO
PROYECTO DE CARRERA INGENIERÍA EN INFORMÁTICA
CATEDRA: LENGUAJES Y COMPILADORES

LOS LENGUAJES DE PROGRAMACIÓN

Profesor:
Félix Márquez

Autores:
Keibel Guilarte
C.I 28.726.602
Oriana Márquez
C.I 31.354.299
Jeanny Monagas
C.I 30.857.471
Victor Vargas
C.I 30.697.219

Ciudad Guayana, junio del 2026.

Resumen Académico

Se analizó la infraestructura lingüística y computacional de las herramientas de desarrollo modernas, abordando el lenguaje de programación como un producto arquitectónico de software sujeto a restricciones formales. Se documenta el fenómeno de convergencia multiparadigma. Para medir el impacto empírico de la arquitectura interna de estos lenguajes, se ejecutó un estudio de benchmarking en procesamiento intensivo, demostrando que para Rust 126,38ms, con un consumo de memoria de 3.3 (MB) asegura un rendimiento ultrarrápido en sistemas de bajo nivel manteniendo estricta seguridad de memoria. Zing 132,1ms, con un consumo de memoria pico 2,6 (MB) funciona como una alternativa moderna y simplificada a C para crear infraestructura crítica, JavaScript 1.824,84ms con un consumo de memoria pico de 40,5 (MB) es el motor interactivo indispensable que le da vida y dinamismo a la web moderna y por último Python 30.843,52ms con un consumo de memoria pico de 9,8 (MB) destaca en backend e inteligencia artificial por su enorme versatilidad y sintaxis legible. Finalmente, se diseñó un Lenguaje de Dominio Específico (DSL), denominado Lenguaje L, es fundamental porque traduce las necesidades eléctricas a instrucciones computacionales. Su eficiencia radica en un conjunto de tokens directos y reglas simples que permiten al analizador procesar comandos a la alta velocidad que exige el control físico en tiempo real de las microrredes. En cuanto a su expresividad, la integración de palabras clave propias del dominio eléctrico permite a los ingenieros modelar escenarios complejos mediante un código altamente legible que refleja el comportamiento del hardware sin ambigüedades. Finalmente, esta precisión estructural maximiza la mantenibilidad del software, ya que facilita la depuración y permite la detección inmediata de errores de sintaxis antes de la ejecución, actuando como una capa de seguridad preventiva e indispensable para evitar daños en la infraestructura física. Orientado al control lógico-físico del sistema ECO-GRID de microrredes.

Introducción.

En el ámbito de la formación inicial en ciencias de la computación, el acercamiento a los lenguajes de programación se caracteriza por adoptar predominantemente la perspectiva del usuario final. Desde este enfoque práctico e instrumental, los lenguajes son concebidos como herramientas estáticas cuya función primordial radica casi exclusivamente en permitir la traducción de la lógica algorítmica hacia una sintaxis particular orientada a la resolución efectiva de problemas. Sin embargo, al trascender esta visión utilitaria y adoptar la rigurosa perspectiva del diseño de compiladores, se produce un cambio de paradigma donde el lenguaje deja de ser un simple medio de expresión para revelarse como una infraestructura arquitectónica compleja y diseñada de manera minuciosa para cumplir objetivos técnicos precisos. Es en este contexto de revisión estructural que el estudio analiza a profundidad cómo las decisiones arquitectónicas fundamentales, específicamente aquellas que operan a nivel morfológico, léxico y sintáctico, impactan de manera directa en los complejos procesos de análisis interno que ejecutan tanto los compiladores como los intérpretes durante el procesamiento del código fuente.

Entre la arquitectura del lenguaje y su ejecución real, la investigación enuncia una exploración total a través de paradigmas divergentes, fundamentándose en el análisis estructural de lenguajes de programación modernos como Zig, Python, Rust y JavaScript. Esta evaluación integral permite contrastar entornos informáticos con distintas filosofías de diseño, observando cómo manejan sus respectivas infraestructuras frente a diversos desafíos. El estudio consolida sus reflexiones mediante la formulación de una propuesta práctica centrada en el desarrollo de un Lenguaje de Dominio Específico (DSL). Es a través de esta conjunción investigativa que se logra evidenciar, de manera concluyente, el impacto de las decisiones de diseño. En última instancia, se demuestra como la precisión formal aplicada durante la construcción de los lenguajes se establece como el factor determinante para garantizar niveles

óptimos de rendimiento y seguridad, variables verdaderamente críticas en entornos de procesamiento intensivo y en sistemas complejos de interfaz hombre-máquina.

Paradigma Imperativo/Estructural

Es como darle instrucciones a alguien que no sabe hacer absolutamente nada por su cuenta. Le tienes que decir paso a paso qué hacer, cómo moverse y en qué momento exacto cambiar las cosas. La memoria de la computadora funciona aquí como una pizarra donde estás borrando y escribiendo a cada rato. El gran problema con esto es que, si cambias un dato sin querer, puedes arruinar otra parte del proceso que dependía de esa información, así que el programador tiene que estar supervisando todo manualmente para que la lógica de la máquina no colapse.

Paradigma Orientado a Objetos (POO)

Aquí la idea es ponerle orden al caos del caso anterior. Imagina que guardas la información en cajas fuertes, lo que se conoce como encapsulamiento; nadie puede tocar los datos que hay adentro a menos que use los botones oficiales y permitidos de la caja. Antes, a los programadores les gustaba hacer "familias" de cajas donde las hijas heredaban todo de los padres, pero el problema era que si el padre fallaba, se rompía toda la cadena. Hoy en día, la tendencia es armar las cosas como si fueran piezas de Lego sueltas que se combinan. De esta forma, al sistema no le importa la etiqueta de la pieza o "qué es", sino "qué sabe hacer", dándote mucha más flexibilidad para armar programas sin que todo se derrumbe.

Paradigma Lógico/Declarativo

Este enfoque es totalmente distinto a los demás, porque aquí tú no le das órdenes a la máquina ni le explicas el paso a paso. Es como hablar con un detective experto: tú simplemente le das las pistas verdaderas, le explicas las reglas del juego y le haces la pregunta final que necesitas resolver. El lenguaje tiene un motor interno que se pone a investigar, conecta los puntos por sí solo usando un proceso llamado unificación, y te entrega la respuesta sin que tengas que

guiarlo. Básicamente, te olvidas del "cómo" se hace el trabajo pesado y te concentras únicamente en pedir el "qué".

Actividad II

Benchmarking comparativo

a) Especificaciones del hardware y entorno

Todas las corridas se ejecutaron sobre la misma máquina, con los procesos de fondo reducidos al mínimo y midiendo únicamente el tiempo de cómputo (excluyendo entrada y salida).

Componente	Detalle
CPU	Intel Core i7-7700T @ 2.90 GHz
Núcleos	4 núcleos físicos / 8 lógicos
Memoria RAM	16 GB DDR4
Sistema operativo	Windows 10 Pro 64 bits (build 19045)
Python	CPython 3.13
JavaScript	Node.js (motor V8)
Rust	Rustc 1.95.0, compilado con --release (LLVM)

Zig	Zig 0.16.0, compilado con -O ReleaseFast (LLVM)
-----	---

b) Algoritmo y metodología

Se calculó la longitud de la secuencia de Collatz para todos los enteros del rango $[2, N)$, con $N = 1.000.000$. La métrica reportada corresponde al tiempo de cómputo puro, sin operaciones de entrada y salida. Para cada lenguaje se ejecutaron tres corridas y se reporta el promedio.

El número con más pasos dentro del rango analizado fue 837.799, que requirió 524 iteraciones hasta converger a 1. Cada implementación se compiló o ejecutó con optimizaciones activadas: cargo build --release en Rust, zig build-exe -O ReleaseFast en Zig, node en JavaScript y python en CPython 3.13.

c) Tabla comparativa de benchmarking

Los lenguajes aparecen ordenados del más rápido al más lento. Los tiempos están expresados en milisegundos y representan el promedio de tres corridas; el consumo de memoria corresponde al pico observado durante la ejecución.

Lenguaje de programación	Paradigma dominante	Mecanismo de ejecución y compilación	Tiempo de ejecución promedio (ms)	Consumo de memoria pico (MB)
Rust	Multiparadigma (funcional, imperativo)	Compilación nativa (LLVM)	126,38	3,3

Lenguaje de programación	Paradigma dominante	Mecanismo de ejecución y compilación	Tiempo de ejecución promedio (ms)	Consumo de memoria pico (MB)
Zig	Imperativo / estructurado	Compilación nativa (LLVM)	132,10	2,6
JavaScript	Multiparadigma (prototípico, funcional)	JIT (Just in Time) / motor V8	1.824,84	40,5
Python	Multiparadigma (orientado a objetos, imperativo)	Interpretado (CPython / VM)	30.843,52	9,8

Actividad III

El equipo Atomic Code denomina ATOM (Automatización Topológica para Operaciones de Microred) a su realización del Lenguaje L. Es un DSL imperativo de alto nivel, con tipado estático y palabras clave en español, diseñado para operadores de planta del sistema ECO-GRID. Se especifica a continuación su léxico, sus palabras reservadas, su gramática en EBNF y dos programas de demostración.

a) Especificación Léxica

El alfabeto del lenguaje se compone de letras del alfabeto latino (a hasta z, A hasta Z), dígitos decimales (0 hasta 9), el guion bajo, los operadores aritméticos y relacionales, los signos de puntuación necesarios y los caracteres en blanco. La siguiente tabla enumera la totalidad de los tokens reconocidos, su patrón regular y un ejemplo válido.

Token	Patrón regular	Ejemplo
IDENT	<code>[a-zA-Z_][a-zA-Z0-9_]{0,31}</code>	<code>bateria_01, temp</code>
ENTERO	<code>[0-9]+</code>	<code>55, 200</code>
REAL	<code>[0-9]+\.[0-9]+</code>	<code>90.5, 0.25</code>
BOOLEANO	<code>verdadero falso</code>	<code>verdadero</code>
CADENA	<code>"[^\n]*"</code>	<code>"Alerta termica"</code>
OP_ARIT	<code>+ - * / %</code>	<code>+</code>
OP_REL	<code>== != < > <= >=</code>	<code>>=</code>
OP_LOG	<code>y o no</code>	<code>y</code>
ASIG	<code>=</code>	<code>=</code>
DELIM	<code>() , ; :</code>	<code>(</code>
COMENTARIO	<code>// [^\n]*</code>	<code>// Observacion</code>

Los identificadores deben iniciar con una letra o guion bajo, admiten letras, dígitos y guion bajo en su cuerpo, son sensibles a mayúsculas, tienen longitud máxima de 32 caracteres y no pueden coincidir con una palabra reservada. Los blancos, tabuladores y saltos de línea se descartan en el análisis léxico, ya que los bloques se delimitan con palabras de cierre explícitas. Solo se admiten comentarios de una sola línea, introducidos con dos barras inclinadas.

b) Palabras Reservadas

- Control de flujo de declaración

El lenguaje cuenta con 24 palabras reservadas para el control de flujo, la declaración de identificadores y los valores booleanos canónicos

<code>inicio</code>	<code>fin</code>	<code>var</code>	<code>const</code>
<code>si_verdadero</code>	<code>entonces</code>	<code>sino</code>	<code>sino_si</code>
<code>fin_si</code>	<code>mientras</code>	<code>ejecutar</code>	<code>fin_mientras</code>
<code>para</code>	<code>desde</code>	<code>hasta</code>	<code>paso</code>

fin_para	retornar	esperar	verdadero
falso	y	o	no

- Primitivas de dominio, sensores de lectura

Primitiva	Retorno	Función
init_grid()	ninguno	Inicializa el bus con los drivers
leer_temperatura(bateria_id)	REAL (°C)	Temperatura de celda de la batería
estado_carga(bateria_id)	REAL (%)	Nivel de carga, de 0 a 100
flujo_entrada(inversor_id)	REAL (kW)	Potencia de entrada del inversor
flujo_salida(inversor_id)	REAL (kW)	Potencia de salida del inversor
generacion_solar()	REAL (kW)	Producción agregada de los paneles
generacion_eolica()	REAL (kW)	Producción agregada de las turbinas
demanda_actual(sector_id)	REAL (kW)	Consumo total o por sector
hora_del_dia()	ENTERO	Hora actual, de 0 a 23

- Primitivas de dominio, actuadores y eventos

Primitiva	Función
conmutar_linea(sector_id, estado)	Conecta o desconecta la línea de un sector
activar_refrigeracion(bateria_id)	Enciende el enfriamiento auxiliar
desactivar_refrigeracion(bateria_id)	Apaga el enfriamiento
aislar_sector(sector_id)	Aísla físicamente un sector del bus
conectar_sector(sector_id)	Reconecta un sector al bus principal

Primitiva	Función
inyectar_red(potencia_kW)	Vende el excedente a la red pública
desviar_consumo(origen, destino)	Redirige el consumo a otra fuente
emitir_alerta(mensaje)	Notificación operativa al panel HMI
emitir_alerta_critica(mensaje)	Notificación de emergencia
registrar_evento(mensaje)	Bitácora persistente del sistema
esperar(segundos)	Pausa la ejecución

- Constantes simbólicas predefinidas.

```
// Estados de rele
CONECTADO, DESCONECTADO

// Fuentes de energia
RED_COMERCIAL, RED_LOCAL, RED_PUBLICA

// Identificadores de sector
sector_industrial, sector_oficinas, sector_medico,
sector_servidores, sector_residencial

// Identificadores de dispositivo predefinidos por la planta
bateria_01 ... bateria_N, bateria_principal,
inversor_01 ... inversor_N, linea_solar, linea_eolica
```

c) Gramática sintáctica formal

La gramática se especifica en notación EBNF y resulta LL(1), lo que habilita un analizador descendente recursivo de complejidad lineal sobre la longitud del programa fuente.

```
(* === Programa principal === *)
programa      ::= "inicio" bloque "fin"
bloque        ::= sentencia*
```

(* === Sentencias === *)

sentencia ::= declaracion | asignacion | condicional
 | bucle_mientras | bucle_para
 | llamada_funcion | retorno

declaracion ::= ("var" | "const") IDENT "=" expresion

asignacion ::= IDENT "=" expresion

(* === Estructuras de control === *)

condicional ::= "si_verdadero" expresion "entonces" bloque
 ("sino_si" expresion "entonces" bloque) *
 ("sino" bloque) ?
 "fin_si"

bucle_mientras ::= "mientras" expresion "ejecutar" bloque "fin_mientras"

bucle_para ::= "para" IDENT "desde" expresion "hasta" expresion
 ("paso" expresion) ? "ejecutar" bloque "fin_para"

(* === Expresiones, precedencia ascendente === *)

expresion ::= expr_logica

expr_logica ::= expr_compar (("y" | "o") expr_compar) *

expr_compar ::= expr_aritm (op_relacional expr_aritm) ?

expr_aritm ::= termino (("+" | "-") termino) *

termino ::= factor (("*" | "/" | "%") factor) *

factor ::= ENTERO | REAL | BOOLEANO | CADENA | IDENT
 | llamada_funcion | "(" expresion ")"
 | "no" factor | "-" factor

op_relacional ::= "==" | "!=" | "<" | ">" | "<=" | ">="

```

llamada_funcion ::= IDENT "(" lista_args? ")"
lista_args      ::= expresion ( "," expresion ) *
retorno        ::= "retornar" expresion?

```

La precedencia de los operadores en orden ascendente es la siguiente.

Nivel	Operadores	Asociatividad
1	o	Izquierda
2	y	Izquierda
3	no	Derecha
4	== != < > <= >=	Izquierda
5	+ - (binario)	Izquierda
6	* / %	Izquierda
7	- (unario)	Derecha
8	() agrupación	Ninguna

d) Programas Operativos.

- Escenario A.

El programa monitoriza de manera iterativa la temperatura de un banco de baterías. Si supera el umbral crítico de 55 grados Celsius, activa el enfriamiento auxiliar, desconecta la línea solar y desvía el consumo industrial hacia la red comercial. Si la condición persiste tras los ciclos máximos permitidos, emite alerta crítica y aísla el banco del bus.

```

// Escenario A: prevencion de fuga termica y alivio de carga
inicio

    const UMBRAL_CRITICO    = 55          // grados Celsius
    const CICLOS_MAXIMOS    = 10
    const PERIODO_MUESTREO = 30          // segundos

```

```

var temperatura_actual = 0
var ciclos_en_peligro  = 0
var sistema_estable    = falso

init_grid()
registrar_evento("Iniciando monitoreo termico de bateria_01")

mientras ciclos_en_peligro < CICLOS_MAXIMOS y no sistema_estable ejecutar
    temperatura_actual = leer_temperatura(bateria_01)

    si_verdadero temperatura_actual > UMBRAL_CRITICO entonces
        emitir_alerta("Temperatura critica: " + temperatura_actual + " C")
        activar_refrigeracion(bateria_01)
        conmutar_linea(linea_solar, DESCONECTADO)
        desviar_consumo(sector_industrial, RED_COMERCIAL)
        ciclos_en_peligro = ciclos_en_peligro + 1
        esperar(PERIODO_MUESTREO)
    sino
        sistema_estable = verdadero
        desactivar_refrigeracion(bateria_01)
        conmutar_linea(linea_solar, CONECTADO)
        desviar_consumo(sector_industrial, RED_LOCAL)
        registrar_evento("Estabilizacion termica alcanzada")
    fin_si
fin_mientras

si_verdadero ciclos_en_peligro >= CICLOS_MAXIMOS entonces
    emitir_alerta_critica("FALLA PERSISTENTE: aislando bateria_01")
    aislar_sector(bateria_01)
sino
    registrar_evento("Proteccion termica finalizada con exito")
fin_si

```

fin

- Escenario B

El programa evalúa el estado del sistema y decide la estrategia. Con carga superior al 90 por ciento y producción solar mayor a la demanda, inyecta el excedente a la red pública. Con carga inferior al 20 por ciento en horario nocturno, aísla los sectores industriales, de oficinas y residenciales para preservar las áreas médicas y de servidores. En cualquier otro caso, mantiene la operación nominal.

```
// Escenario B: balance de carga y optimizacion energetica autonoma
inicio

    const UMBRAL_OPTIMO      = 90    // % de carga
    const UMBRAL_CRITICO     = 20    // % de carga
    const HORA_NOCHE_INICIO  = 19
    const HORA_NOCHE_FIN     = 6

    var carga_bateria        = 0
    var solar                 = 0
    var consumo               = 0
    var excedente             = 0
    var hora                  = 0
    var es_horario_nocturno  = falso

    init_grid()

    carga_bateria            = estado_carga(bateria_principal)
    solar                     = generacion_solar()
    consumo                   = demanda_actual()
    hora                      = hora_del_dia()
    es_horario_nocturno      = (hora >= HORA_NOCHE_INICIO) o (hora <= HORA_NOCHE_FIN)
```

```
si_verdadero carga_bateria > UMBRAL_OPTIMO y solar > consumo entonces
    excedente = solar + generacion_eolica() - consumo
    conmutar_linea(RED_PUBLICA, CONECTADO)
    inyectar_red(excedente)
    emitir_alerta("Inyectando " + excedente + " kW a la red publica")

sino_si carga_bateria < UMBRAL_CRITICO y es_horario_nocturno entonces
    emitir_alerta_critica("Carga critica nocturna, aislamiento selectivo")
    aislar_sector(sector_industrial)
    aislar_sector(sector_oficinas)
    aislar_sector(sector_residencial)
    conectar_sector(sector_medico)
    conectar_sector(sector_servidores)

sino
    registrar_evento("Operacion nominal")
fin_si
fin
```

Conclusiones

La síntesis de la teoría arquitectónica de los lenguajes, los resultados del benchmarking empírico y el desarrollo práctico del lenguaje ATOM demuestra de forma definitiva que las decisiones de diseño morfológico, léxico y sintáctico determinan de manera absoluta el rendimiento, la seguridad y la viabilidad operativa de un sistema. La teoría postula que un lenguaje no es solo sintaxis, sino una infraestructura donde el diseño del compilador o intérprete dicta su capacidad en entornos de procesamiento intensivo. El benchmarking empírico valida esta premisa al someter a diferentes lenguajes al cálculo continuo de la secuencia de Collatz. Esto demuestra prácticamente que la concentración de alto nivel caracteriza a los lenguajes interpretados y sacrifica el rendimiento frente a las optimizaciones nativas de hardware.

Mientras lenguajes de propósito general como Rust y Zig optimizan la ejecución a nivel de máquina, el diseño del Lenguaje de Dominio Específico ATOM ilustra cómo la teoría de compiladores se aplica para garantizar seguridad semántica en un grupo operativo crítico: la red ECO-GRID. Al establecer una gramática formal estrictamente $LL(1)$, ATOM asegura que su analizador descendente recursivo opere sin ambigüedades y con una complejidad lineal sobre la longitud del programa fuente. Esta decisión de diseño sintáctico es vital para procesar de manera rápida y predecible el código escrito por los operadores, especialmente al ejecutar rutinas de emergencia como la prevención de fugas térmicas en baterías o el aislamiento de sectores de la red ante sobrecargas. Adicionalmente, sus reglas léxicas, que integran un tipado estático, bloques delimitados por palabras de cierre explícitas y palabras clave en español. En conjunto, el benchmarking refleja el costo estructural de los paradigmas de ejecución, mientras que la creación de ATOM evidencia el valor de adaptar la dureza gramatical a

las exigencias operativas de un entorno industrial, reafirmando que el diseño de un lenguaje dicta fatalmente el comportamiento y la seguridad del software en el mundo real.

Bibliografia

IBM. (2023). *What is Object-Oriented Programming (OOP)?* IBM Data and AI.

<https://www.ibm.com/topics/object-oriented-programming>

Massachusetts Institute of Technology. (s.f.). *Computer Language Engineering: Lexical and Syntax Analysis*. MIT OpenCourseWare. <https://ocw.mit.edu/courses/6-035-computer-language-engineering-spring-2010/>

National Renewable Energy Laboratory. (s.f.). *Microgrids*. NREL.gov.

<https://www.nrel.gov/grid/microgrids.html>

Python Software Foundation. (2026). *The Python Language Reference*. Python.org.

<https://docs.python.org/3/reference/>

Stanford University. (s.f.). *Compilers: Top-Down Parsing and LL(1)*. Stanford Computer Science. <https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/>

The Rust Project Developers. (2026). *The Rust Reference*. Rust-lang.org. <https://doc.rust-lang.org/reference/>

V8 Project. (2024). *V8 JavaScript engine: Architecture and JIT Compilation*. V8.dev.

<https://v8.dev/docs>

Zig Software Foundation. (2026). *Zig Language Reference*. Ziglang.org.

<https://ziglang.org/documentation/master/>